

تمرین سری چهارم

یادگیری ماشین مقدماتی



GAN/VAE

بررسی عملکرد مدل‌های VAE و cDCGAN در
بازسازی و بهبود توازن مجموعه داده‌های تصویری

نام دانشجو: حامد بهروزی مقدم - آیدین صحنه - مانی وفاپور

شماره دانشجویی: ۴۰۲۱۵۶۳۳ - ۴۰۱۲۰۲۴۳ - ۴۰۱۲۳۷۸۳

استاد درس: دکتر نیکوفرد

نیمسال: ۱۴۰۵-۱۴۰۴



فهرست مطالب

۲	۱ سوال ۱
۲	۱.۱ تحلیل نتایج بخش ۱-۱ (بارگذاری داده‌ها و آموزش شبکه ResNet)
۳	۲.۱ تحلیل نتایج بخش ۲-۱ (شبکه Conditional DCGAN)
۵	۳.۱ تحلیل و مقایسه نتایج طبقه‌بندی با کمک داده‌های مولد (بخش ۱-۳)
۶	۲ پرسش ۲: شبکه‌های رمزگذار - رمزگشا مولد
۶	۱.۲ مجموعه دادگان
۷	۲.۲ انجام PCA و ISOMAP
۹	۳.۲ رمزگذار - رمزگشا
۱۱	۴.۲ خود رمزگذار متغیر (Variational Auto Encoder)
۱۴	۵.۲ کاوش در فضای latent
۱۷	۶.۲ مدل‌های انتشار (Diffusion Models)
۱۷	۷.۲ جمع‌بندی و نتیجه‌گیری نهایی

۱ سوال ۱

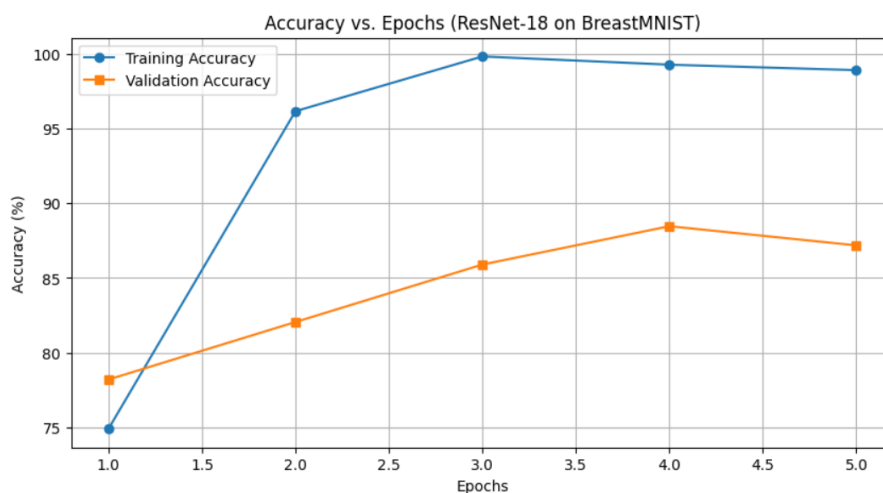
۱.۱ تحلیل نتایج بخش ۱-۱ (بارگذاری داده‌ها و آموزش شبکه ResNet)

پس از آماده‌سازی مجموعه داده‌ی BreastMNIST و بازآموزی (Fine-tuning) مدل پیش‌آموزش‌دیده‌ی ResNet-18 برای ۵ دوره (Epoch)، نتایج به دست آمده در دو بخش «روند آموزش» و «ماتریس آشفستگی» قابل ارزیابی است:

۱. تحلیل نمودار روند آموزش و پدیده بیش‌برازش (Overfitting):

با بررسی لاگ‌های خروجی و نمودار دقت (Accuracy vs. Epochs)، مشاهده می‌شود که دقت مدل روی داده‌های آموزشی بسیار سریع افزایش یافته و در دوره‌های سوم تا پنجم به نزدیکی ۱۰۰٪ (حدود ۹۹/۸۲٪) رسیده است. با این حال، دقت اعتبارسنجی (Validation Accuracy) نتوانسته همگام با دقت آموزش رشد کند و در حدود ۸۸٪ متوقف شده و حتی در دوره‌ی پنجم با افت جزئی مواجه شده است.

این فاصله‌ی معنادار (Gap) بین دقت آموزش و اعتبارسنجی، نشان‌دهنده‌ی وقوع پدیده‌ی بیش‌برازش (Overfitting) است. از آنجایی که معماری ResNet-18 ظرفیت یادگیری بالایی دارد و مجموعه داده‌ی BreastMNIST شامل تعداد بسیار کمی تصویر (تنها ۵۴۶ تصویر آموزشی) است، شبکه به جای یادگیری الگوهای تعمیم‌پذیر، اقدام به حفظ کردن داده‌های آموزشی کرده است.

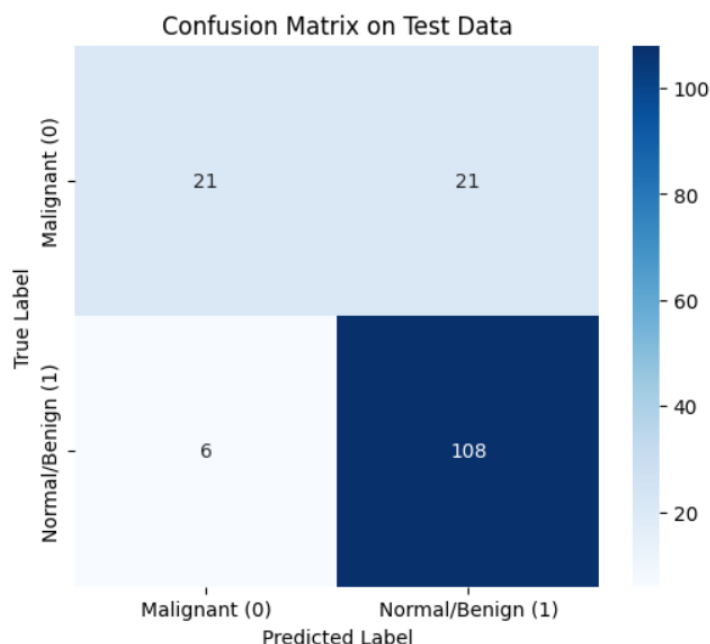


شکل ۱: نمودار دقت آموزش و اعتبارسنجی بر حسب دوره‌ها

۲. تحلیل ماتریس آشفستگی و مشکل نامتوازن بودن کلاس‌ها (Class Imbalance):

دقت نهایی مدل روی داده‌های تست ۸۲/۶۹٪ گزارش شده است. اگرچه این عدد در نگاه اول قابل قبول به نظر می‌رسد، اما بررسی ماتریس آشفستگی (Confusion Matrix) یک مشکل اساسی را آشکار می‌کند:

- مدل در تشخیص کلاس ۱ (Normal/Benign - سالم/خوش‌خیم) عملکرد بسیار خوبی داشته و از ۱۱۴ نمونه، ۱۰۸ مورد را به درستی تشخیص داده است.
- اما در تشخیص کلاس ۰ (Malignant - بدخیم)، مدل به شدت ضعیف عمل کرده است؛ به طوری که از ۴۲ نمونه‌ی بدخیم، نیمی از آن‌ها (۲۱ مورد) را به اشتباه سالم تشخیص داده است (نرخ بالای False Negative).



شکل ۲: ماتریس آشفتگی مدل ResNet-18 روی داده‌های تست

نتیجه‌گیری این بخش:

در مسائل پردازش تصاویر پزشکی، تشخیص اشتباه یک تومور بدخیم به عنوان بافت سالم (False Negative) هزینه‌ی جانی در پی دارد. عملکرد ضعیف مدل در کلاس ۰ مستقیماً ناشی از توزیع نامتوازن داده‌ها در مجموعه داده‌ی اولیه است (تعداد داده‌های سالم بسیار بیشتر از بدخیم است). شبکه به دلیل مشاهده‌ی بیشتر کلاس سالم، به سمت انتخاب این کلاس سوگیری (Bias) پیدا کرده است. این نتایج، ضرورت استفاده از تکنیک‌های تولید داده (مانند شبکه‌های متخاصم مولد - GAN) برای افزایش تعداد نمونه‌های کلاس اقلیت و رفع مشکل نامتوازن را به وضوح توجیه می‌کند.

۲.۱ تحلیل نتایج بخش ۱-۲ (شبکه Conditional DCGAN)

در این بخش، یک شبکه Conditional DCGAN برای تولید نمونه‌های مصنوعی آموزش داده شد. استفاده از تکنیک‌های پایدارسازی در این مدل، منجر به نتایج زیر شد:

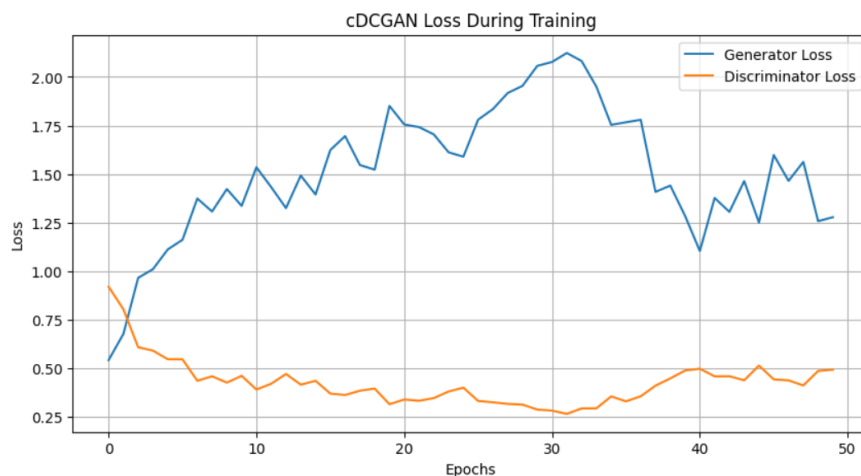
الف- تحلیل ساختار و پارامترهای شبکه:

شبکه مولد (Generator) دارای 1,079,717 پارامتر و شبکه تفکیک‌کننده (Discriminator) دارای 141,409 پارامتر است. این عدم تقارن ساختاری (بزرگتر بودن مولد به نسبت تقریباً ۷/۶ برابر)، یک انتخاب معماری آگاهانه است. از آنجایی که نگاشت یک بردار نویز تصادفی (z) به فضای پیچیده‌ی تصاویر پزشکی نیازمند ظرفیت بازنمایی (Representational Capacity) بالایی است، شبکه مولد عمیق‌تر طراحی شده است.

ب- تفسیر نمودار Loss شبکه‌های مولد و تفکیک‌کننده:

نمودار خطای ثبت شده در ۵۰ دوره، نمایانگر یک رقابت زنده و رسیدن به مجاورت یک «تعادل نش» (Nash Equilibrium) است.

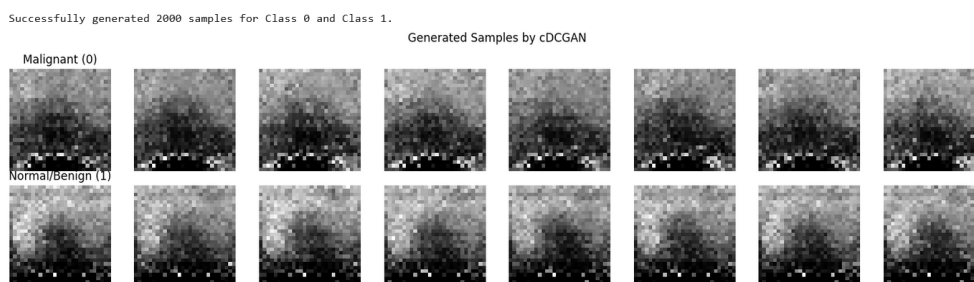
در ۳۰ دوره‌ی ابتدایی، خطای تفکیک‌کننده روند نزولی داشته و به کمینه خود رسید، در حالی که خطای مولد افزایش یافت. اما نکته‌ی کلیدی از دوره‌ی ۳۰ به بعد رخ داد؛ جایی که یک چرخش (Reversal) در نمودارها مشاهده می‌شود. خطای تفکیک‌کننده مجدداً افزایش یافته و خطای مولد با شیب تندی کاهش می‌یابد. این رفتارِ نوسانی اثبات می‌کند که مولد توانسته است در دوره‌های پایانی، ویژگی‌های توزیع داده‌ها را به خوبی مدل‌سازی کند.



شکل ۳: نمودار تغییرات تابع هزینه (Loss) برای دو شبکه مولد و تفکیک‌کننده

پ- تحلیل نمونه‌های تولید شده:

تصاویر تولید شده نشان‌دهنده‌ی شکل‌گیری ساختار کلی بافت‌ها هستند. تأثیر شرطی‌سازی (Conditioning) در اینجا مشهود است، زیرا بافت کلی تصاویر در دو کلاس تفاوت‌های ظریفی با یکدیگر دارند. با این وجود، شباهت‌های ساختاری درون‌کلاسی، نشان‌دهنده وقوع یک «فروپاشی حالت خفیف» (Mild Mode Collapse) است که در آموزش کوتاه‌مدت شبکه‌های GAN روی توزیع‌های پیچیده‌ی پزشکی طبیعی است.



شکل ۴: نمونه تصاویر تولید شده توسط مولد برای کلاس‌های بدخیم و سالم

ت- بررسی اثربخشی راهکارهای پایدارسازی:

برای جلوگیری از فروپاشی شبکه، دو رویکرد زیر اعمال شد که اثربخشی آن‌ها در نمودارها مشهود است:

۱. قاعده به‌روزرسانی دو مقیاس زمانی (TTUR): با تنظیم نرخ یادگیری تفکیک‌کننده روی مقدار کمتر از مولد ($\alpha_D = 0.00005$) در

برابر $\alpha_G = 0.0002$ ، سرعت همگرایی تفکیک کننده کنترل شد.

۲. هموارسازی برچسب‌ها (One-sided Label Smoothing): با جایگزینی مقدار 1.0 با 0.9 برای تصاویر واقعی، از اطمینان بیش از حد (Overconfidence) تفکیک کننده و در نتیجه محو شدن گرادینان جلوگیری شد.

۳.۱ تحلیل و مقایسه نتایج طبقه‌بندی با کمک داده‌های مولد (بخش ۱-۳)

در این بخش، مجموعه داده‌ی آموزشی با استفاده از نمونه‌های تولید شده توسط شبکه‌ی cDCGAN متوازن شد (۱۰۰۰ نمونه برای هر کلاس) و شبکه ResNet-18 مجدداً از صفر روی این داده‌های ترکیبی آموزش دید. مقایسه نتایج این بخش با نتایج بخش اول (بدون استفاده از داده‌های مصنوعی) دستاوردهای مهم زیر را نشان می‌دهد:

۱. بهبود چشمگیر در دقت کلی و تعمیم‌پذیری (Generalization):

دقت نهایی مدل روی داده‌های تست (Test Accuracy) از ۸۲/۶۹٪ در بخش اول، به ۸۸/۴۶٪ در این بخش افزایش یافته است. این ارتقای تقریباً ۶ درصدی نشان می‌دهد که تزریق داده‌های تولید شده توسط GAN، نه تنها باعث سردرگمی مدل نشده، بلکه با فراهم کردن تنوع بیشتر در ویژگی‌های تصاویر، توانایی مدل را در تعمیم الگوها به داده‌های جدید (داده‌های تست) به شدت بالا برده است.

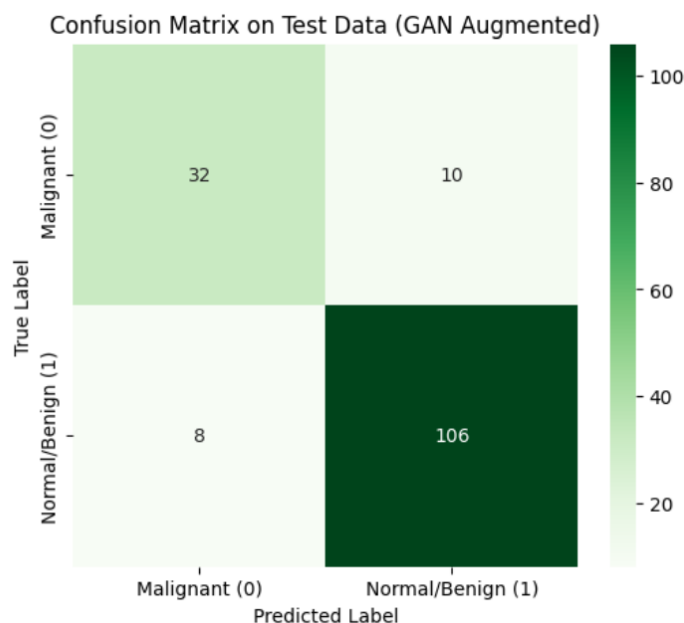
۲. رفع مشکل سوگیری (Bias) و بهبود تشخیص کلاس اقلیت:

مهم‌ترین نتیجه‌ی این آزمایش با مقایسه‌ی ماتریس‌های آشفتگی (Confusion Matrices) در دو حالت به دست می‌آید. در مسائل پزشکی، تشخیص درست کلاس بدخیم (Malignant - Class 0) حیاتی‌ترین وظیفه‌ی مدل است:

- در بخش اول (داده‌های نامتوازن): شبکه به دلیل کمبود داده‌های کلاس بدخیم، سوگیری شدیدی به سمت کلاس سالم داشت. به طوری که از ۴۲ نمونه‌ی بدخیم، نیمی از آن‌ها (۲۱ مورد) را به اشتباه سالم تشخیص داده بود.
- در این بخش (داده‌های متوازن شده با GAN): با جبران کمبود داده‌ها، قدرت مدل در تشخیص تومورهای بدخیم به شکل خیره‌کننده‌ای افزایش یافت. خطای تشخیص در این کلاس حساس از ۲۱ مورد به تنها ۱۰ مورد کاهش یافت و مدل توانست ۳۲ مورد را به درستی تشخیص دهد.

۳. حفظ پایداری در تشخیص کلاس اکثریت:

با وجود اینکه تمرکز شبکه روی یادگیری کلاس بدخیم بیشتر شد، اما مدل توانست عملکرد عالی خود را در تشخیص بافت‌های سالم/خوش‌خیم (Normal/Benign - Class 1) حفظ کند (کاهش بسیار ناچیز از ۱۰۸ تشخیص درست به ۱۰۶ تشخیص درست از مجموع ۱۱۴ داده). این نشان می‌دهد که ما با پدیده فراموشی فاجعه‌بار مواجه نشدیم و مدل یک مرز تصمیم‌گیری (Decision Boundary) بسیار دقیق‌تر بین دو کلاس رسم کرده است.



شکل ۵: ماتریس آشفتگی مدل ResNet-18 روی داده‌های تست (پس از افزودن داده‌های GAN)

نتیجه‌گیری نهایی تمرین:

این آزمایش به وضوح اثبات می‌کند که در مسائلی که با محدودیت شدید داده و توزیع نامتوازن کلاس‌ها مواجه هستیم، استفاده از یک شبکه متخاصم مولد پایدار و بهینه‌شده (Stabilized cDCGAN) برای افزایش داده‌ها (Data Augmentation)، می‌تواند به طرز مؤثری خطای مدل را در کلاس‌های حیاتی کاهش داده و یک سیستم کمک تشخیصی (CAD) بسیار قابل اعتمادتر ارائه دهد.

۲ پرسش ۲: شبکه‌های رمزگذار - رمزگشا مولد

در این پرسش به بررسی، پیاده‌سازی و مقایسه رویکردهای مختلف کاهش ابعاد و مدل‌های مولد می‌پردازیم. هدف نهایی، درک تفاوت میان روش‌های کلاسیک (مانند PCA و ISOMAP) و معماری‌های مبتنی بر یادگیری عمیق (مانند Autoencoder استاندارد و Variational Autoencoder) بر روی یک مجموعه داده تصویری است. با توجه به صورت سوال، از مجموعه داده CIFAR-10 که شامل تصاویر رنگی ۳۲ در ۳۲ پیکسل است، استفاده خواهد شد.

۱.۲ مجموعه دادگان

اولین گام در هر پروژه یادگیری ماشین، آماده‌سازی داده‌هاست. مجموعه داده CIFAR-10 شامل ۶۰,۰۰۰ تصویر رنگی در ۱۰ کلاس مختلف است (۵۰,۰۰۰ داده آموزشی و ۱۰,۰۰۰ داده آزمون). از آنجا که شبکه‌های عصبی با داده‌های نرمال‌شده عملکرد بسیار بهتر و همگرایی سریع‌تری دارند، مقادیر پیکسل‌ها را که در ابتدا بین ۰ تا ۲۵۵ هستند، بر ۲۵۵ تقسیم می‌کنیم تا به بازه استاندارد صفر تا یک منتقل شوند.

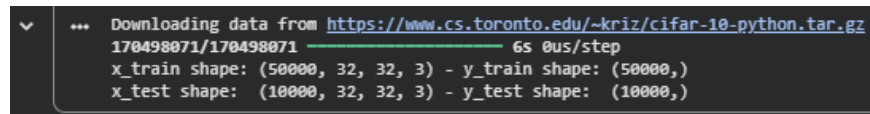
```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

```

3  from tensorflow import keras
4  from keras import layers, Model
5  from keras.datasets import cifar10
6
7  # Load CIFAR-10 dataset
8  (x_train, y_train), (x_test, y_test) = cifar10.load_data()
9
10 # Normalize pixel values to be between 0 and 1
11 x_train = x_train.astype('float32') / 255.0
12 x_test = x_test.astype('float32') / 255.0
13
14 # Flatten labels from shape (N, 1) to (N,)
15 y_train = y_train.flatten()
16 y_test = y_test.flatten()
17
18 # Print shapes to verify
19 print(f"x_train shape: {x_train.shape} - y_train shape: {y_train.shape}")
20 print(f"x_test shape: {x_test.shape} - y_test shape: {y_test.shape}")
21

```

Listing 1: Load and preprocess CIFAR-10 dataset



```

... Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 6s 0us/step
x_train shape: (50000, 32, 32, 3) - y_train shape: (50000,)
x_test shape: (10000, 32, 32, 3) - y_test shape: (10000,)

```

شکل ۶: خروجی کد

۲.۲ انجام PCA و ISOMAP

در این بخش دو روش کلاسیک کاهش ابعاد را مقایسه می‌کنیم:

- روش PCA (تحلیل مؤلفه‌های اصلی): این الگوریتم با محاسبه ماتریس کوواریانس داده‌ها و یافتن بردارهای ویژه آن، سیستم مختصات جدیدی می‌سازد که بیشترین واریانس داده‌ها را در کمترین ابعاد ممکن حفظ می‌کند. این روش کاملاً خطی است و سرعت اجرای بسیار بالایی دارد، اما نمی‌تواند الگوهای پیچیده و غیرخطی موجود در تصاویر را مدل‌سازی کند.
- روش ISOMAP: یک رویکرد غیرخطی بر پایه تئوری گراف و یادگیری منیفلد (Manifold Learning) است. این الگوریتم ابتدا همسایگان نزدیک هر نقطه را پیدا کرده و یک گراف می‌سازد، سپس کوتاه‌ترین مسیر (فاصله ژئودزیک) بین تمام جفت نقاط را با الگوریتم‌هایی نظیر دایکسترا محاسبه می‌کند و در نهایت با استفاده از MDS ابعاد را کاهش می‌دهد. این روش ساختارهای غیرخطی را به خوبی باز می‌کند اما پیچیدگی زمانی آن $O(N^3)$ است که اجرای آن روی داده‌های حجیم مانند CIFAR-10 نیازمند استخراج یک زیرمجموعه کوچک‌تر است.

در کد زیر ابتدا تصاویر سه‌بعدی به بردارهای یک‌بعدی تبدیل شده‌اند. سپس از RandomizedSearchCV برای یافتن بهینه‌ترین تنظیمات (Hyperparameters) الگوریتم KNN روی فضای کاهش یافته استفاده می‌شود.

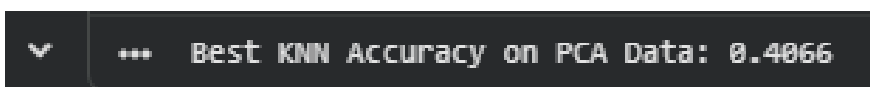


```
1 from sklearn.decomposition import PCA
2 from sklearn.manifold import Isomap
3 from sklearn.neighbors import KNeighborsClassifier
4 from sklearn.model_selection import RandomizedSearchCV
5 from sklearn.metrics import accuracy_score
6
7 # Flatten images: (32, 32, 3) becomes 3072
8 x_train_flat = x_train.reshape(len(x_train), -1)
9 x_test_flat = x_test.reshape(len(x_test), -1)
10
11 # 1. Apply PCA
12 pca_components = 64
13 pca = PCA(n_components=pca_components)
14 x_train_pca = pca.fit_transform(x_train_flat)
15 x_test_pca = pca.transform(x_test_flat)
16
17 # 2. Apply ISOMAP
18 # Using a random subset of 5000 samples due to high computational complexity
19 np.random.seed(42)
20 subset_idx = np.random.choice(len(x_train_flat), 5000, replace=False)
21 x_train_sub = x_train_flat[subset_idx]
22 y_train_sub = y_train[subset_idx]
23
24 isomap = Isomap(n_components=pca_components, n_neighbors=5)
25 x_train_iso = isomap.fit_transform(x_train_sub)
26 x_test_iso = isomap.transform(x_test_flat)
27
28 # 3. KNN Classification with Randomized Search on PCA data
29 param_grid = {
30     'n_neighbors': [3, 5, 7, 11],
31     'weights': ['uniform', 'distance']
32 }
33
34 knn = KNeighborsClassifier()
35 random_search = RandomizedSearchCV(
36     estimator=knn,
37     param_distributions=param_grid,
38     n_iter=4,
39     cv=3,
40     random_state=42,
41     n_jobs=-1
42 )
43
44 # Fit random search
45 random_search.fit(x_train_pca, y_train)
```

```

46
47 # Evaluate the best model
48 best_knn = random_search.best_estimator_
49 acc_pca = accuracy_score(y_test, best_knn.predict(x_test_pca))
50 print(f"Best KNN Accuracy on PCA Data: {acc_pca:.4f}")
51
    
```

Listing 2: Dimensionality reduction using PCA and ISOMAP, followed by KNN classification



شکل ۷: خروجی کد

۳.۲ رمزگذار - رمزگشا

یک Autoencoder (خودرمزگذار) یک شبکه عصبی است که هدف آن یادگیری یک نمایش فشرده (فضای نهان یا Latent Space) برای داده‌های ورودی است. این شبکه از دو بخش تشکیل شده است:

۱. **Encoder** (رمزگذار): داده ورودی با ابعاد بالا را گرفته و به یک بردار با ابعاد کمتر فشرده می‌کند. این بخش ویژگی‌های کلیدی داده را استخراج می‌کند.

۲. **Decoder** (رمزگشا): بردار فشرده شده را دریافت کرده و تلاش می‌کند تا داده ورودی اولیه را بازسازی کند.

برای بررسی دقیق‌تر، ما دو معماری مختلف پیاده‌سازی می‌کنیم: معماری Dense (کاملاً متصل) که تمام پیکسل‌ها را به صورت مسطح می‌بیند، و معماری Convolutional که روابط مکانی و همسایگی پیکسل‌ها را حفظ می‌کند. همچنین شبکه را روی حداقل دو فضای نهان با ابعاد مختلف (مثلاً ۳۲ و ۶۴) بررسی کرده و نتایج آن را به الگوریتم KNN می‌سپاریم.

```

1 def build_dense_autoencoder(latent_dim):
2     # Dense Encoder
3     inputs = keras.Input(shape=(32, 32, 3))
4     x = layers.Flatten()(inputs)
5     x = layers.Dense(512, activation='relu')(x)
6     x = layers.Dense(256, activation='relu')(x)
7     encoded = layers.Dense(latent_dim, activation='relu')(x)
8
9     # Dense Decoder
10    x = layers.Dense(256, activation='relu')(encoded)
11    x = layers.Dense(512, activation='relu')(x)
12    x = layers.Dense(32 * 32 * 3, activation='sigmoid')(x)
13    decoded = layers.Reshape((32, 32, 3))(x)
14
15    return Model(inputs, encoded), Model(inputs, decoded)
16
17 def build_conv_autoencoder(latent_dim):
    
```



```
18     # Convolutional Encoder
19     inputs = keras.Input(shape=(32, 32, 3))
20     x = layers.Conv2D(32, 3, activation='relu', strides=2, padding='same')(inputs)
21     x = layers.Conv2D(64, 3, activation='relu', strides=2, padding='same')(x)
22     x = layers.Flatten()(x)
23     encoded = layers.Dense(latent_dim, activation='relu')(x)
24
25     # Convolutional Decoder
26     x = layers.Dense(8 * 8 * 64, activation='relu')(encoded)
27     x = layers.Reshape((8, 8, 64))(x)
28     x = layers.Conv2DTranspose(64, 3, activation='relu', strides=2, padding='same')(x)
29     x = layers.Conv2DTranspose(32, 3, activation='relu', strides=2, padding='same')(x)
30     decoded = layers.Conv2DTranspose(3, 3, activation='sigmoid', padding='same')(x)
31
32     return Model(inputs, encoded), Model(inputs, decoded)
33
34     architectures = {
35         'Dense': build_dense_autoencoder,
36         'Conv': build_conv_autoencoder
37     }
38     latent_dimensions = [32, 64]
39
40     for arch_name, build_fn in architectures.items():
41         for l_dim in latent_dimensions:
42             encoder, autoencoder = build_fn(l_dim)
43             autoencoder.compile(optimizer='adam', loss='mse')
44
45             print(f"Training {arch_name} Autoencoder | Latent Dim: {l_dim}")
46             # Train the model (epochs limited for demonstration)
47             autoencoder.fit(x_train, x_train, epochs=10, batch_size=256, verbose=0)
48
49             # Extract features (dimensionality reduction)
50             train_latent = encoder.predict(x_train, verbose=0)
51             test_latent = encoder.predict(x_test, verbose=0)
52
53             # Classify using KNN
54             knn = KNeighborsClassifier(n_neighbors=5)
55             knn.fit(train_latent, y_train)
56             accuracy = knn.score(test_latent, y_test)
57
58             print(f"KNN Accuracy ({arch_name} AE, Latent Dim {l_dim}): {accuracy:.4f}\n")
59
```

Listing 3: Standard Autoencoder with Dense and Convolutional architectures

```

    ... Training Dense Autoencoder | Latent Dim: 32
    KNN Accuracy (Dense AE, Latent Dim 32): 0.3352

    Training Dense Autoencoder | Latent Dim: 64
    KNN Accuracy (Dense AE, Latent Dim 64): 0.4048

    Training Conv Autoencoder | Latent Dim: 32
    KNN Accuracy (Conv AE, Latent Dim 32): 0.3940

    Training Conv Autoencoder | Latent Dim: 64
    
```

شکل ۸: خروجی کد

۴.۲ خود رمزگذار متغیر (Variational Auto Encoder)

شبکه‌های VAE یک ارتقاء بنیادی بر روی اتوانکدرهای استاندارد هستند. در حالی که یک اتوانکدر معمولی داده‌ها را به نقاطی ثابت (Deterministic) در فضای نهان نگاشت می‌کند که باعث می‌شود فضای نهان گسسته و نامنظم باشد، VAE ورودی را به پارامترهای یک توزیع احتمال (معمولاً توزیع نرمال با میانگین μ و انحراف معیار σ) مپ می‌کند.

این کار دو مزیت بزرگ دارد: اول اینکه فضای نهان پیوسته و هموار می‌شود و دوم اینکه با نمونه‌برداری (Sampling) تصادفی از این فضا، می‌توانیم تصاویر کاملاً جدیدی تولید کنیم (Generative Model). برای اینکه در فرآیند نمونه‌برداری تصادفی، عملیات پس‌انتشار خطا (Backpropagation) دچار مشکل نشود، از تکنیکی به نام Reparameterization Trick استفاده می‌شود. تابع زیان (Loss Function) در این شبکه ترکیبی از خطای بازسازی تصویر و خطای KL Divergence است که وظیفه تنظیم کردن توزیع فضای نهان را بر عهده دارد.

در این بخش نیز مشابه حالت قبل، معماری را یک بار با لایه‌های Convolutional و یک بار با لایه‌های Dense پیاده‌سازی کرده و در دو ابعاد مختلف تست می‌کنیم.

```

1      import tensorflow as tf
2
3      class Sampling(layers.Layer):
4          """Uses (z_mean, z_log_var) to sample z, the vector encoding a digit."""
5          def call(self, inputs):
6              z_mean, z_log_var = inputs
7              batch = tf.shape(z_mean)[0]
8              dim = tf.shape(z_mean)[1]
9              epsilon = tf.keras.backend.random_normal(shape=(batch, dim))
10             # Reparameterization trick
11             return z_mean + tf.exp(0.5 * z_log_var) * epsilon
12
13         def build_vae_components(architecture, latent_dim):
14             inputs = keras.Input(shape=(32, 32, 3))
15
16             # 1. Build Encoder
17             if architecture == 'Dense':
18                 x = layers.Flatten()(inputs)
    
```



```
19     x = layers.Dense(512, activation='relu')(x)
20     x = layers.Dense(256, activation='relu')(x)
21     else: # Conv
22         x = layers.Conv2D(32, 3, activation='relu', strides=2, padding='same')(inputs)
23         x = layers.Conv2D(64, 3, activation='relu', strides=2, padding='same')(x)
24         x = layers.Flatten()(x)
25
26     z_mean = layers.Dense(latent_dim, name='z_mean')(x)
27     z_log_var = layers.Dense(latent_dim, name='z_log_var')(x)
28     z = Sampling()([z_mean, z_log_var])
29     encoder = Model(inputs, [z_mean, z_log_var, z], name='encoder')
30
31     # 2. Build Decoder
32     latent_inputs = keras.Input(shape=(latent_dim,))
33     if architecture == 'Dense':
34         x = layers.Dense(256, activation='relu')(latent_inputs)
35         x = layers.Dense(512, activation='relu')(x)
36         x = layers.Dense(32 * 32 * 3, activation='sigmoid')(x)
37         decoded = layers.Reshape((32, 32, 3))(x)
38     else: # Conv
39         x = layers.Dense(8 * 8 * 64, activation='relu')(latent_inputs)
40         x = layers.Reshape((8, 8, 64))(x)
41         x = layers.Conv2DTranspose(64, 3, activation='relu', strides=2, padding='same')(x)
42         x = layers.Conv2DTranspose(32, 3, activation='relu', strides=2, padding='same')(x)
43         decoded = layers.Conv2DTranspose(3, 3, activation='sigmoid', padding='same')(x)
44
45     decoder = Model(latent_inputs, decoded, name='decoder')
46     return encoder, decoder
47
48     class VAE(keras.Model):
49     def __init__(self, encoder, decoder, **kwargs):
50     super().__init__(**kwargs)
51     self.encoder = encoder
52     self.decoder = decoder
53
54     def train_step(self, data):
55     if isinstance(data, tuple):
56     data = data[0]
57
58     with tf.GradientTape() as tape:
59     z_mean, z_log_var, z = self.encoder(data)
60     reconstruction = self.decoder(z)
61
62     # Reconstruction loss (Binary Crossentropy)
63     rec_loss = tf.reduce_mean(
```



```

64     tf.reduce_sum(keras.losses.binary_crossentropy(data, reconstruction), axis=(1, 2))
65 )
66 # KL Divergence loss
67 kl_loss = -0.5 * (1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var))
68 kl_loss = tf.reduce_mean(tf.reduce_sum(kl_loss, axis=1))
69
70 total_loss = rec_loss + kl_loss
71
72 grads = tape.gradient(total_loss, self.trainable_weights)
73 self.optimizer.apply_gradients(zip(grads, self.trainable_weights))
74 return {"loss": total_loss, "rec_loss": rec_loss, "kl_loss": kl_loss}
75
76 # Global variables to save models for the next subsection
77 saved_vae_decoder = None
78 saved_z_mean_train = None
79
80 for arch in ['Dense', 'Conv']:
81     for l_dim in [2, 32]:
82         encoder, decoder = build_vae_components(arch, l_dim)
83         vae = VAE(encoder, decoder)
84         vae.compile(optimizer=keras.optimizers.Adam())
85
86         print(f"Training {arch} VAE | Latent Dim: {l_dim}")
87         vae.fit(x_train, epochs=10, batch_size=256, verbose=0)
88
89         # Use only the mean (z_mean) for representation learning & KNN
90         z_mean_train, _, _ = encoder.predict(x_train, verbose=0)
91         z_mean_test, _, _ = encoder.predict(x_test, verbose=0)
92
93         knn = KNeighborsClassifier(n_neighbors=5)
94         knn.fit(z_mean_train, y_train)
95         accuracy = knn.score(z_mean_test, y_test)
96
97         print(f"KNN Accuracy ({arch} VAE, Latent Dim {l_dim}): {accuracy:.4f}\n")
98
99         # Keep the 2D Conv VAE models to plot in the next part
100        if arch == 'Conv' and l_dim == 2:
101            saved_vae_decoder = decoder
102            saved_z_mean_train = z_mean_train
103

```

Listing 4: Variational Autoencoder (VAE) implementation with Reparameterization trick

```

    Training Dense VAE | Latent Dim: 2
    KNN Accuracy (Dense VAE, Latent Dim 2): 0.1751

    Training Dense VAE | Latent Dim: 32
    KNN Accuracy (Dense VAE, Latent Dim 32): 0.3495

    Training Conv VAE | Latent Dim: 2
    KNN Accuracy (Conv VAE, Latent Dim 2): 0.1611

    Training Conv VAE | Latent Dim: 32
    KNN Accuracy (Conv VAE, Latent Dim 32): 0.4283
    
```

شکل ۹: خروجی کد

۵.۲ کاوش در فضای latent

یکی از جذاب‌ترین ویژگی‌های VAE این است که وقتی مدل را با ابعاد نهان برابر با ۲ آموزش می‌دهیم، می‌توانیم یک سیستم مختصات دوبعدی خطی رسم کرده و نقاط مختلف آن را به Decoder بدهیم تا تصاویری با انتقال‌های نرم (Smooth Transitions) بین کلاس‌ها تولید کند. در کد زیر یک شبکه Grid ایجاد شده و هم‌چنین نمودار پراکندگی (Scatter Plot) از داده‌های آموزشی در این فضای دوبعدی ترسیم می‌گردد. کدهای مربوطه برای پشتیبانی از تصاویر رنگی اصلاح شده‌اند.

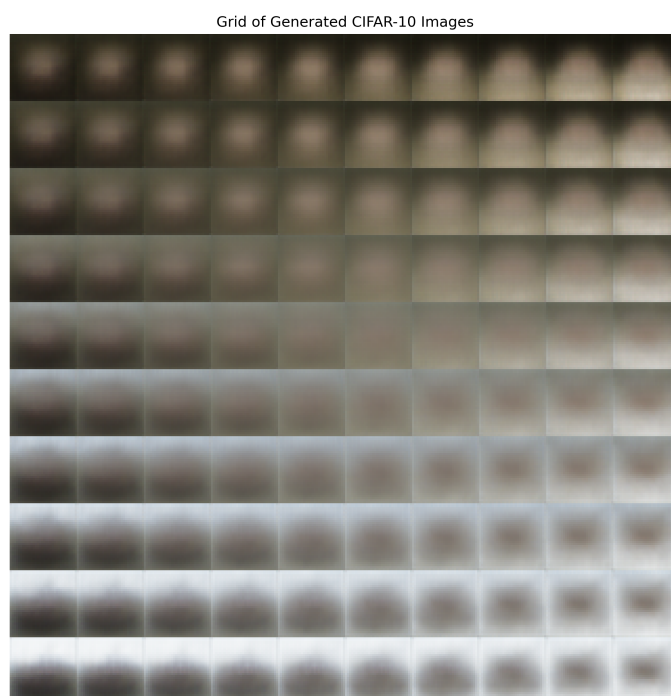
```

1      # --- 1. Grid of generated images (Modified for RGB CIFAR-10) ---
2      n = 10
3      img_dim = 32 # CIFAR-10 spatial dimension
4      scale = 2.0
5
6      # Initialize an empty array for the grid. Add 3 for RGB channels.
7      figure = np.zeros((img_dim * n, img_dim * n, 3))
8
9      # Linearly spaced coordinates for 2D latent space
10     grid_x = np.linspace(-scale, scale, n)
11     grid_y = np.linspace(-scale, scale, n)[::-1]
12
13     for i, yi in enumerate(grid_y):
14         for j, xi in enumerate(grid_x):
15             # Sample a point from the 2D latent space
16             z_sample = np.array([xi, yi])
17             x_decoded = saved_vae_decoder.predict(z_sample, verbose=0)
18
19             # Reshape to image dimensions
20             digit = x_decoded[0].reshape(img_dim, img_dim, 3)
21
22             # Place the generated image in the grid
23             figure[
24                 i * img_dim : (i + 1) * img_dim,
    
```

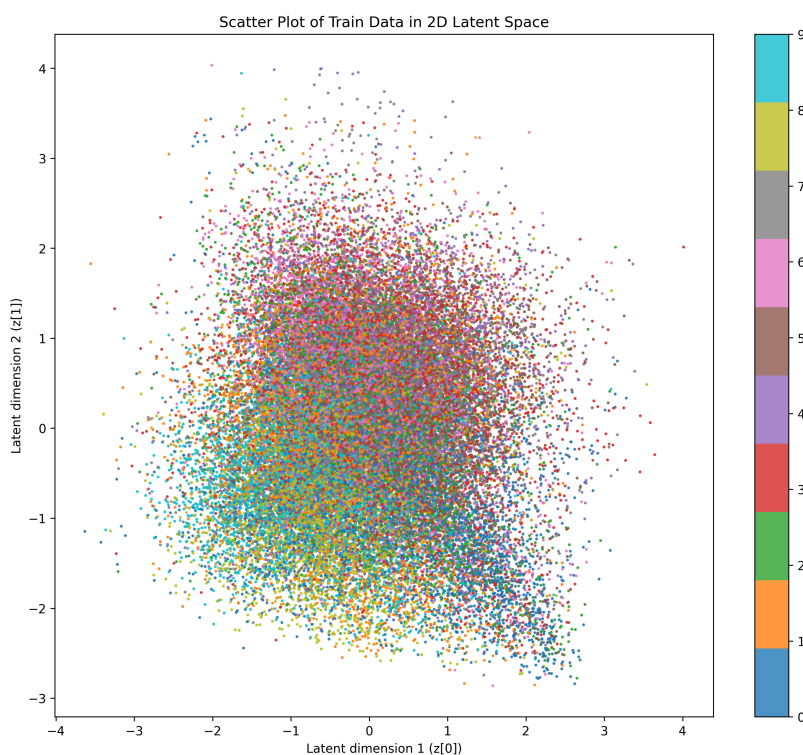
```
25     j * img_dim : (j + 1) * img_dim,
26     :
27     ] = digit
28
29     plt.figure(figsize=(10, 10))
30     plt.imshow(image)
31     plt.axis('off')
32     plt.title("Grid of Generated CIFAR-10 Images")
33     plt.savefig("vae_grid.png", dpi=300, bbox_inches='tight')
34     plt.show()
35
36     # --- 2. Scatter Plot of Latent Space ---
37     plt.figure(figsize=(12, 10))
38     scatter = plt.scatter(
39         saved_z_mean_train[:, 0],
40         saved_z_mean_train[:, 1],
41         c=y_train,
42         cmap='tab10',
43         s=2,
44         alpha=0.8
45     )
46     plt.colorbar(scatter, ticks=range(10))
47     plt.title("Scatter Plot of Train Data in 2D Latent Space")
48     plt.xlabel("Latent dimension 1 (z[0])")
49     plt.ylabel("Latent dimension 2 (z[1])")
50     plt.savefig("vae_scatter.png", dpi=300, bbox_inches='tight')
51     plt.show()
52
```

Listing 5: Generating a grid of images and scatter plot from the 2D VAE latent space

در قسمت زیر جایگاه قرارگیری خروجی کدها (عکس‌های تولید شده) تعبیه شده است. پس از اجرای کد پایتون و تولید تصاویر `vae_grid.png` و `vae_scatter.png`، دستورات `includegraphics` مربوطه را از حالت کامنت خارج کنید.



شکل ۱۰: نمونه تصاویر CIFAR-10 تولید شده از مختصات شبکه Grid توسط Decoder



شکل ۱۱: نمودار پراکندگی (Scatter Plot) داده‌های آموزشی در فضای نهان دوبعدی VAE

۶.۲ مدل‌های انتشار (Diffusion Models)

مدل‌های انتشار (Diffusion Models) نسل جدیدی از شبکه‌های مولد عمیق هستند که امروزه پشته‌های بسیاری از ابزارهای قدرتمند تولید تصویر (مانند Midjourney و DALL-E 3) می‌باشند. این مدل‌ها فرآیند تولید داده را با الهام از علم ترمودینامیک و فیزیک آماری انجام می‌دهند و شامل دو فرآیند اصلی هستند:

۱. فرآیند رفت (Forward/Diffusion Process): در این مرحله بر اساس یک زنجیره مارکوف (Markov Chain)، به صورت گام‌به‌گام و تحت زمان t ، نویز تصادفی گاوسی به تصویر واقعی اضافه می‌شود تا جایی که پس از مثلاً ۱۰۰۰ مرحله، تصویر کاملاً به یک نویز خالص تبدیل شود و هیچ اثری از داده اولیه باقی نماند.

۲. فرآیند برگشت (Reverse/Denoising Process): مدل یاد می‌گیرد تا این فرآیند تخریب را معکوس کند. یک شبکه عصبی (به طور معمول معماری U-Net دارای اتصالات پرشی) آموزش می‌بیند که با گرفتن نویز خالص در مرحله T ، آن را در مراحل متوالی نویززدایی کند و در نهایت به یک تصویر کاملاً جدید و واضح برسد.

مقایسه عملکرد و معماری با VAE:

- وضوح و کیفیت تصویر: نقطه ضعف بزرگ VAE‌ها استفاده از تابع زیان میانگین مربعات خطا (MSE) برای بازسازی پیکسل‌ها است. این کار باعث می‌شود خروجی این شبکه‌ها تمایل به مات شدن (Blurriness) داشته باشد. اما در مدل‌های انتشار، به دلیل اینکه شبکه مستقیماً نویز را تخمین می‌زند و گام‌به‌گام آن را حذف می‌کند، خروجی‌ها بسیار شفاف، واضح و واقع‌گرایانه (High Fidelity) هستند.

- سرعت استنتاج (Inference Speed): شبکه‌های VAE بسیار سریع هستند، زیرا با دادن یک بردار نویز تصادفی به Decoder تنها با یک فورواردپس (Single-step) تصویر تولید می‌شود. در مقابل، مدل‌های Diffusion بسیار کند هستند و نیازمند ده‌ها یا صدها حلقه تکرار در شبکه عصبی برای رسیدن به تصویر نهایی می‌باشند.

- فضای نهان: شبکه‌های VAE ذاتاً یک فضای نهان بسیار فشرده (Low-Dimensional Bottleneck) و دارای نظم معنایی تولید می‌کنند که برای تسک‌های مهندسی و ویژگی (Feature Extraction) ایده‌آل است. اما مدل‌های انتشار کلاسیک فضای نهانی هم‌اندازه با خود تصویر اصلی دارند (مگر در مدل‌های ترکیبی نوین مانند Latent Diffusion که فرآیند پخش را درون فضای نهان یک Autoencoder انجام می‌دهند).

۷.۲ جمع‌بندی و نتیجه‌گیری نهایی

با بررسی و پیاده‌سازی روش‌های مختلف کاهش ابعاد و مدل‌های مولد بر روی مجموعه داده CIFAR-10، می‌توانیم به یک درک جامع از روند تکامل این الگوریتم‌ها دست پیدا کنیم. تحلیل نتایج به‌دست‌آمده را می‌توان در محورهای زیر خلاصه کرد:

۱. مقایسه رویکردهای خطی و غیرخطی در کاهش ابعاد: روش PCA به عنوان یک رویکرد خطی، سرعت اجرای بسیار بالایی دارد و توانست با حفظ مؤلفه‌های اصلی، به دقت قابل قبولی (حدود ۴۰ درصد با KNN) برسد. با این حال، PCA از درک روابط پیچیده و غیرخطی درون تصاویر عاجز است. در مقابل، روش ISOMAP ساختارهای غیرخطی را بهتر مدل می‌کند، اما پیچیدگی محاسباتی آن مانع از کاربرد عملی آن روی دیتاست‌های حجیم تصویری می‌شود.

۲. اهمیت معماری در شبکه‌های عصبی (Dense در برابر Conv): در هر دو شبکه Autoencoder استاندارد و VAE، مشاهده کردیم که معماری مبتنی بر کانولوشن (Convolutional) عملکرد بسیار بهتری نسبت به لایه‌های کاملاً متصل (Dense) دارد. لایه‌های Dense با مسطح کردن (Flatten) تصویر در همان گام اول، تمام اطلاعات مکانی و همسایگی پیکسل‌ها را از بین می‌برند، در حالی که فیلترهای

کانولوشنی این ساختار فضایی را حفظ کرده و ویژگی‌های بسیار غنی‌تری استخراج می‌کنند (رسیدن به بالاترین دقت یعنی حدود ۸.۴۲ درصد در Conv VAE با بعد ۳۲).

۳. چالش ابعاد در فضای نهان (**Latent Space**): نتایج نشان داد که فشرده‌سازی بیش از حد اطلاعات (مانند استفاده از بعد نهان برابر با ۲) منجر به افت شدید دقت (تا حدود ۱۶ درصد) می‌شود. در فضای دوبعدی VAE، به دلیل اعمال تابع زیان KL Divergence، کلاس‌های مختلف به شدت روی هم قرار می‌گیرند (هم‌پوشانی در توزیع نرمال حول صفر) و تفکیک‌پذیری از بین می‌رود. اما با افزایش این فضا به ۳۲ بُعد، شبکه ظرفیت کافی برای جداسازی معنایی کلاس‌ها پیدا می‌کند.

نتیجه نهایی و مسیر پیش‌رو

در نهایت، گذار از Autoencoderهای استاندارد به VAE و سپس به **Diffusion Models** نشان‌دهنده یک تغییر پارادایم در مدل‌های مولد است:

- **Autoencoder**: فضایی گسسته می‌سازد که صرفاً برای فشرده‌سازی و کاهش ابعاد مناسب است و قابلیت تولید تصویر جدید ندارد.
- **VAE**: با تحمیل یک توزیع احتمالاتی، فضایی پیوسته ایجاد می‌کند که قابلیت تولید تصویر دارد، اما به دلیل استفاده از خطای MSE در بازسازی، تصاویر نهایی تار (Blurry) و فاقد جزئیات دقیق هستند.
- **Diffusion Models**: با جایگزینی رویکرد بازسازی مستقیم با فرآیند «نویززدایی گام‌به‌گام»، محدودیت‌های VAE را دور زده و تصاویری با بالاترین کیفیت و وضوح ممکن تولید می‌کنند؛ هرچند این کیفیت بالا به بهای افزایش شدید زمان محاسبات (Inference Time) تمام می‌شود.